



Traffic Tunnelling & Overlay Networks

Redes de Comunicações II

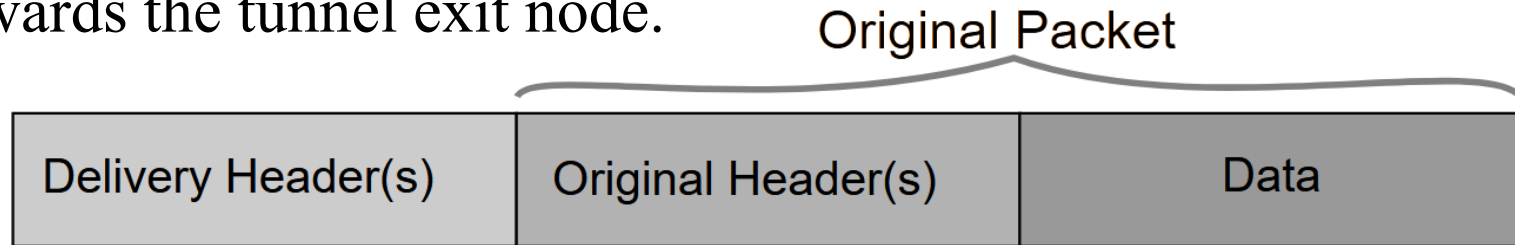
Licenciatura em Engenharia de
Computadores e Informática

Prof. Amaro de Sousa (asou@ua.pt)

DETI-UA, 2025/2026

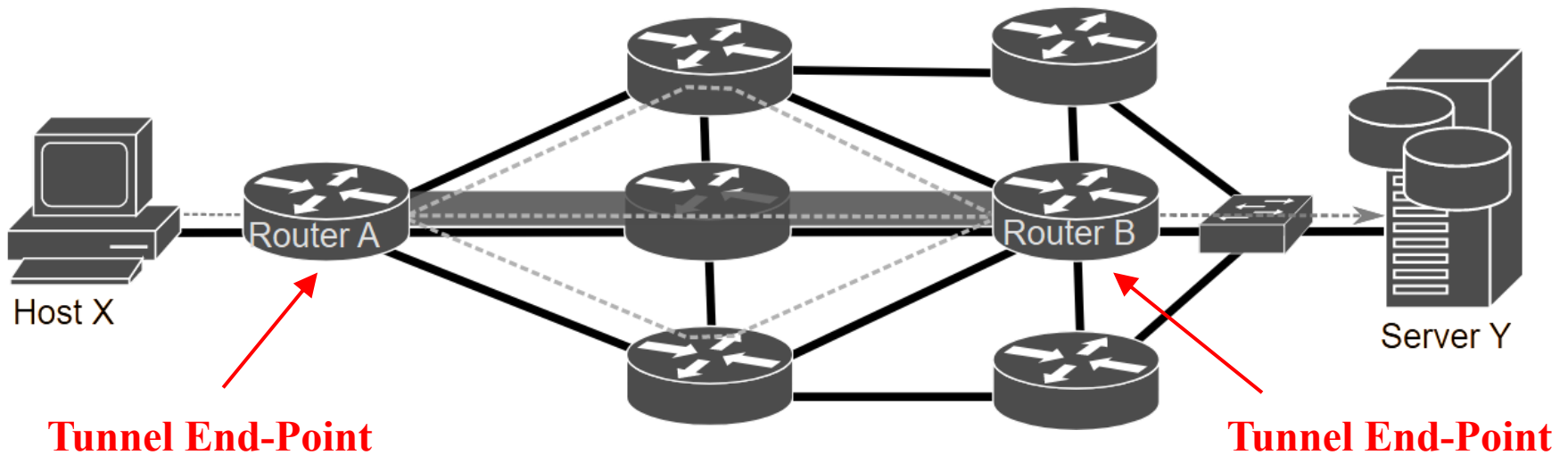
Traffic Tunnel Concept

- A tunnel is implemented by adding (at the tunnel entry node) one or more headers to the original IP packets used to forward the packets towards the tunnel exit node.

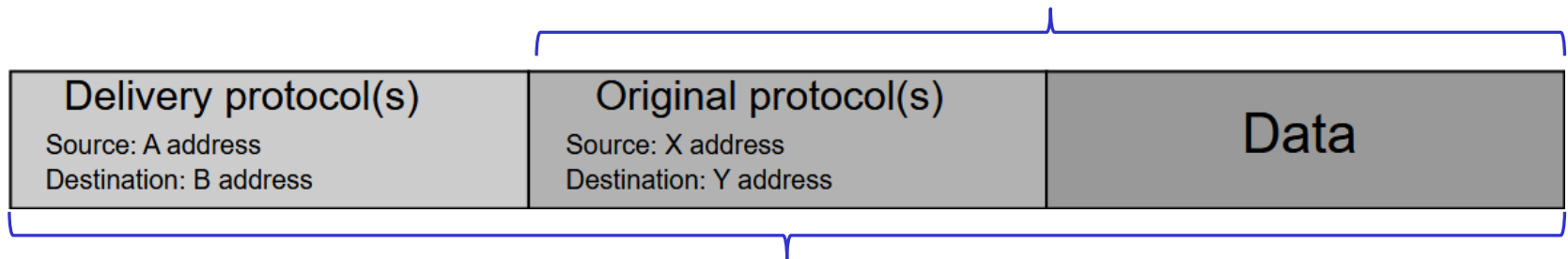


- Some purposes of traffic tunnelling:
 - To guarantee that the IP packets for a given destination network are routed through a specific network node in their routing path towards the destination.
 - To guarantee the forwarding of IP packets to a remote node when the intermediary nodes do not support the original packet network protocol
 - IPv6 packets forwarded through IPv4 networks
 - IPv4 packets forwarded through IPv6 networks
 - To define virtual channels to different traffic flows to provide differentiated Quality of Service, security and/or optimized routing.

Tunnel End-Points



Original Packet from Host X to Server Y:
sent from Host X to Router A
sent from Router B to Server Y



Tunnel packet sent:
from Router A (the entry Tunnel End-Point) to Router B (the exit Tunnel End-Point)

Virtual Tunnel Interface (VTI)

- A VTI is a logical construction that creates a virtual network interface in a network equipment that can be handled as any other network interface.
- Conceptually, a tunnel does not require to have any network addresses other than the ones already bound to the end-point router.
- However, most implementations impose that a network address must be assigned to a VTI to enable IP processing on the interface.
 - The VTI may have an explicitly assigned network address (required to be able to apply routing policies to the traffic router through the tunnel)
 - The VTI may reuse an address of another interface already configured on the router.

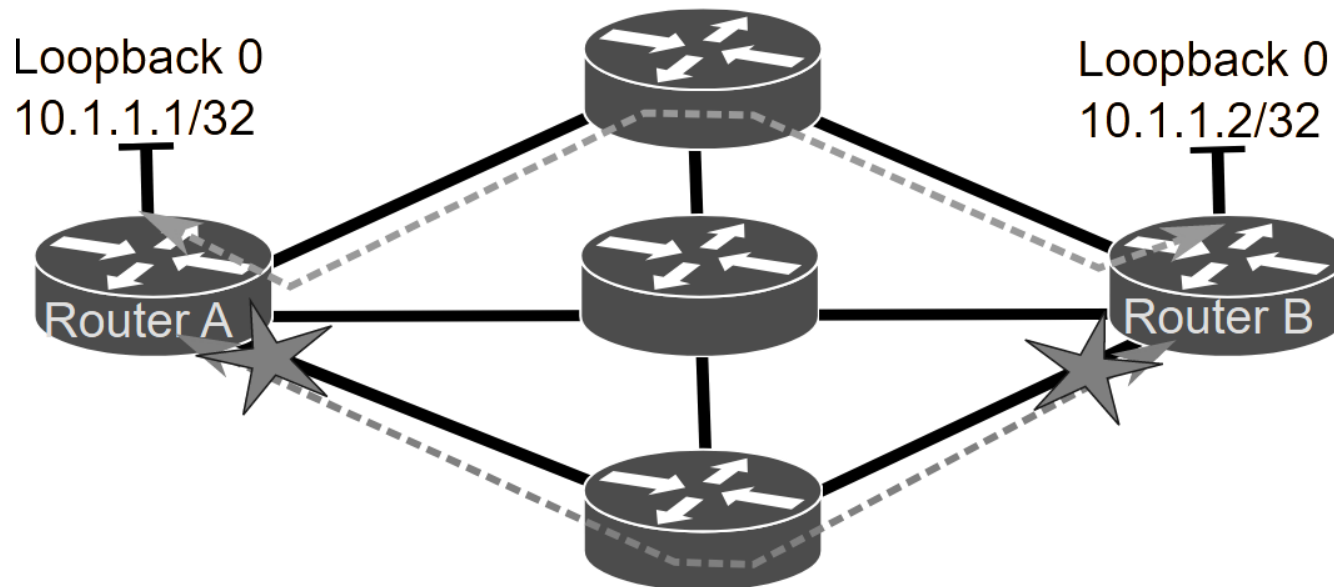
VTI Configuration

- A numeric identifier
- An assigned IP address, to enable IP processing
 - the router adds the tunnel interface to the routing table and allows routing via the interface
- A defined mode or type of tunnel
- Tunnel source, defined as the name of the local interface or IPv4/IPv6 address depending on the type of the tunnel.
- Tunnel destination, defined as a domain name or IPv4/IPv6 address depending on the type of the tunnel.
 - not mandatory in all types of tunnels (in some cases, the tunnel destination end-point is determined dynamically).
- Optionally, it may have additional configurations for routing, security and QoS purposes.

```
1 #interface Tunnel 1
2 #ip address 10.1.1.1 255.255.255.252
3 #ipv6 address 2001:A:A::1/64
4 #ip unnumbered FastEthernet0/0
5 #ipv6 unnumbered FastEthernet0/0
6 #ip ospf cost 10
7 #ipv6 ospf 1 area 0
8 #tunnel mode ipip
9 #tunnel source FastEthernet0/0
10 #tunnel destination 200.2.2.2
```

Loopback Interfaces as Tunnel End-Points

- A loopback interface is another logical construction that creates a virtual network interface completely independent from the other physical and logical router network interfaces.
 - The main propose of a loopback interface is to provide a network address to serve as router identifier in remote network configurations and distributed algorithms.
- The main advantage of using loopback interfaces as tunnel end-points is to obtain a tunnel not bounded to any physical network interface that may fail.



Types of Point-to-Point IP Tunnels

IPv4-IPv4

- Original IPv4 packets are delivered using IPv4 as network protocol.

IPv6-IPv4

- Original IPv6 packets are delivered using IPv4 as network protocol.

GRE IPv4

- The original packets protocol is indicated in the GRE (Generic Routing Encapsulation) header and packets are delivered using IPv4 as network protocol.

IPv4-IPv6

- Original IPv4 packets are delivered using IPv6 as network protocol.

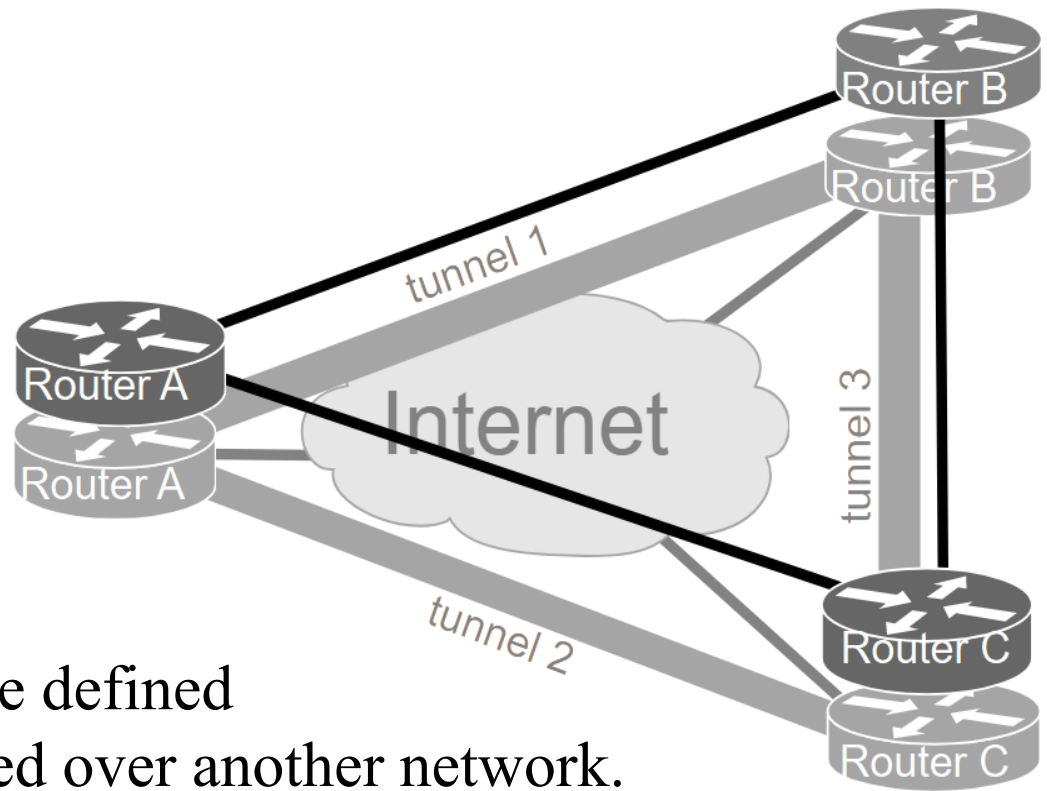
IPv6-IPv6

- Original IPv6 packets are delivered using IPv6 as network protocol.

GRE IPv6

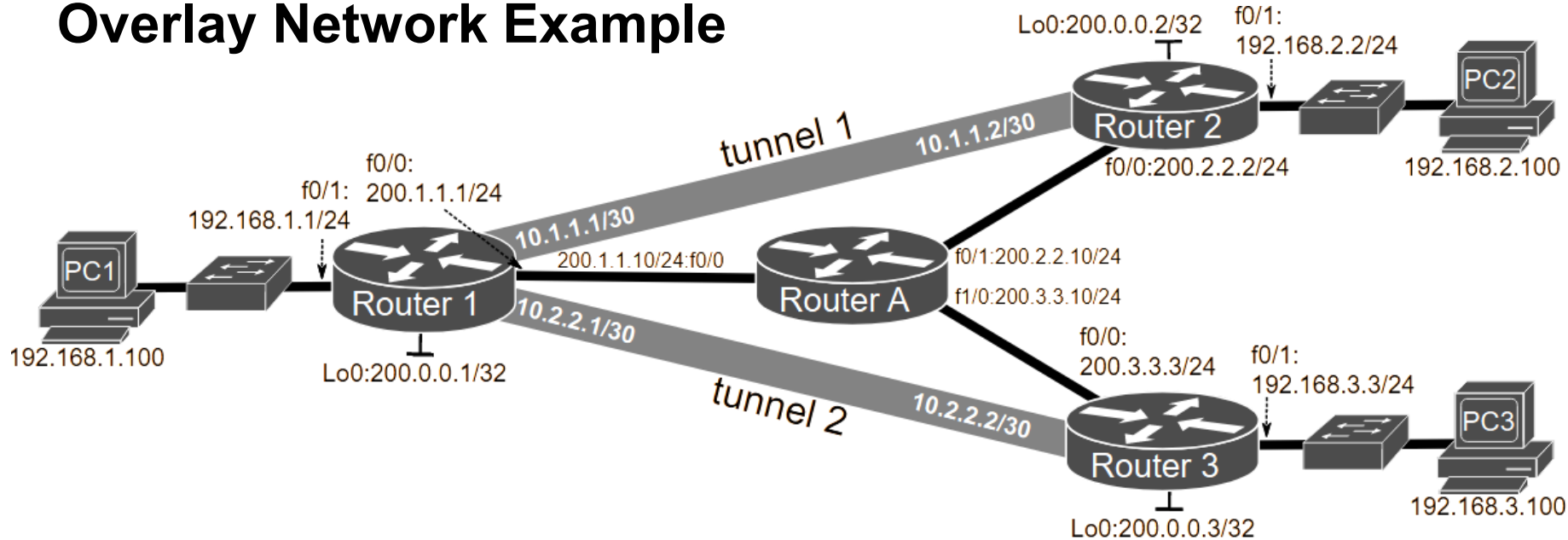
- The original packets protocol is indicated in the GRE header and packets are delivered using IPv6 as network protocol.

Overlay Network



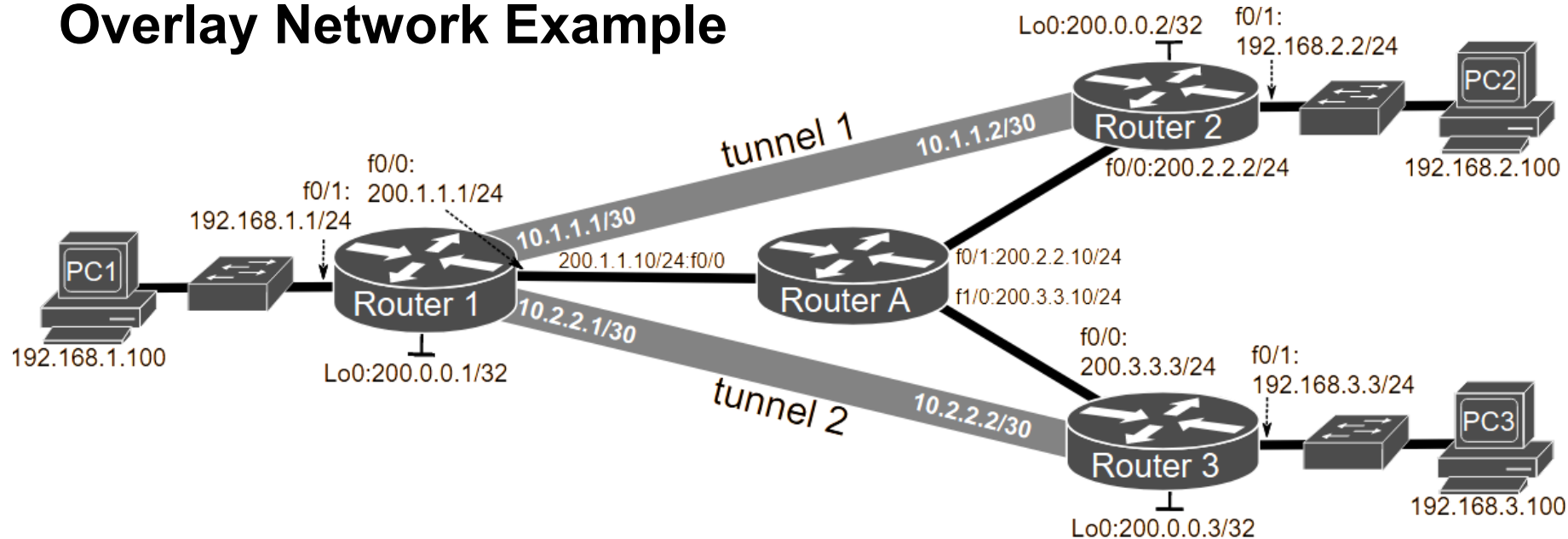
- An overlay network can be defined as a virtual network defined over another network.
 - For a specific purpose like private transport/routing policies, QoS, security.
- The underlying network can be physical or also virtual.
 - May result in multiple layers of overlay networks.
- When any level of privacy protocol is present on an overlay network, it is named a Virtual Private Network (VPN).

Overlay Network Example



- Consider the example of the figure where:
 - The public part of the network is composed by networks 200.1.1.0/24, 200.2.2.0/24, 200.3.3.0/24 and loopback addresses (routing given by OSPF)
 - The private networks are 192.168.1.0/25, 192.168.2.0/24 and 192.168.3.0/24
 - The aim is to setup an overlay network composed by the point-to-point tunnels 1 and 2 to support the communications between private networks

Overlay Network Example



- Before setting up the tunnels:

Router 1

```

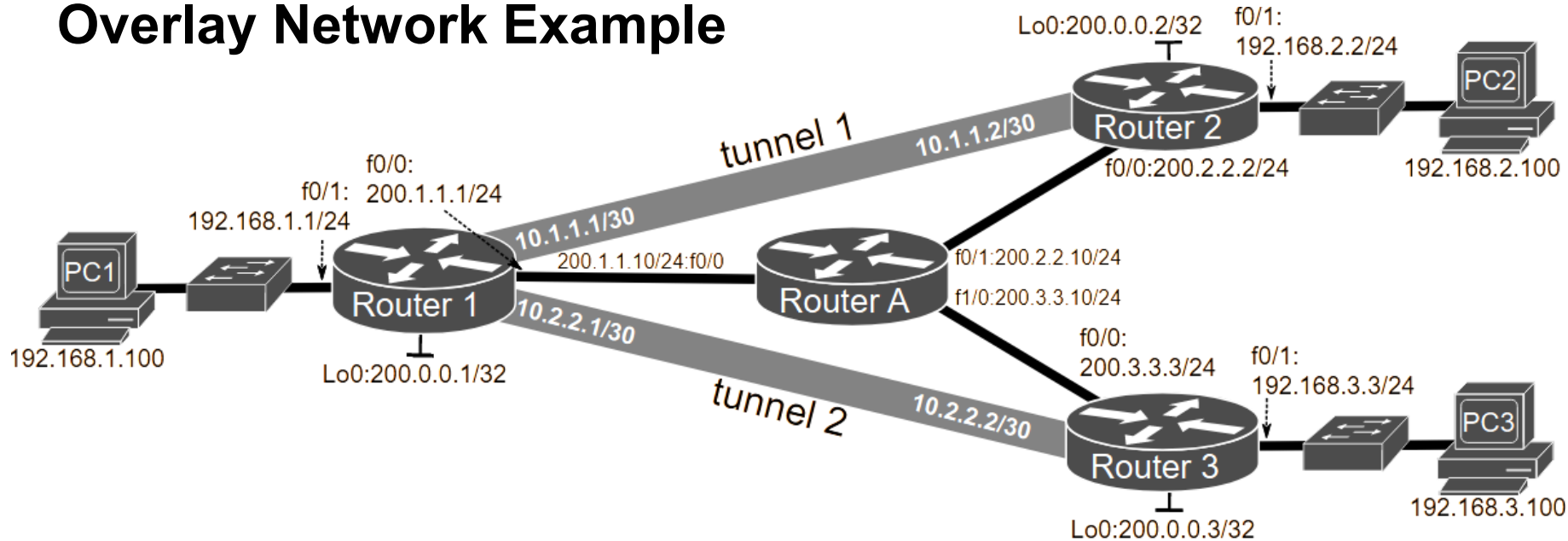
C 192.168.1.0/24 is directly connected, F0/1
O 200.0.0.2 [110/3] via 200.1.1.10, F0/0
O 200.0.0.3 [110/3] via 200.1.1.10, F0/0
C 200.1.1.0/24 is directly connected, F0/0
O 200.2.2.0/24 [110/2] via 200.1.1.10, F0/0
O 200.3.3.0/24 [110/2] via 200.1.1.10, F0/0
  
```

Router A

```

O 200.0.0.1 [110/2] via 200.1.1.1, F0/0
O 200.0.0.2 [110/2] via 200.2.2.2, F0/1
O 200.0.0.3 [110/2] via 200.3.3.3, F1/0
C 200.1.1.0/24 is directly connected, F0/0
C 200.2.2.0/24 is directly connected, F0/1
C 200.3.3.0/24 is directly connected, F1/0
  
```

Overlay Network Example



- Set up tunnel 1:

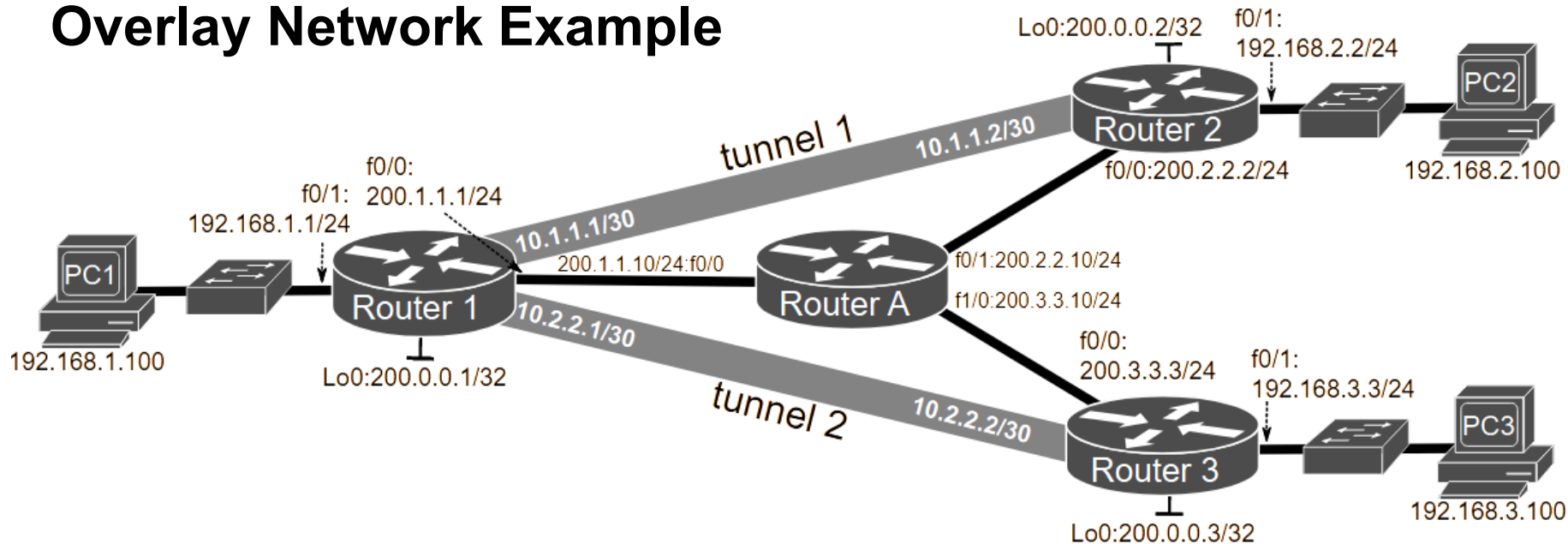
Router 1

```
Router1(config)# interface tunnel 1
Router1(config-if)# tunnel source Loopback0
Router1(config-if)# tunnel destination 200.0.0.2
Router1(config-if)# tunnel mode ipip
Router1(config-if)# ip addr 10.1.1.1 255.255.255.252
```

Router 2

```
Router2(config)# interface tunnel 1
Router2(config-if)# tunnel source Loopback0
Router2(config-if)# tunnel destination 200.0.0.1
Router2(config-if)# tunnel mode ipip
Router2(config-if)# ip addr 10.1.1.2 255.255.255.252
```

Overlay Network Example



- After setting up tunnel 1 and tunnel 2:

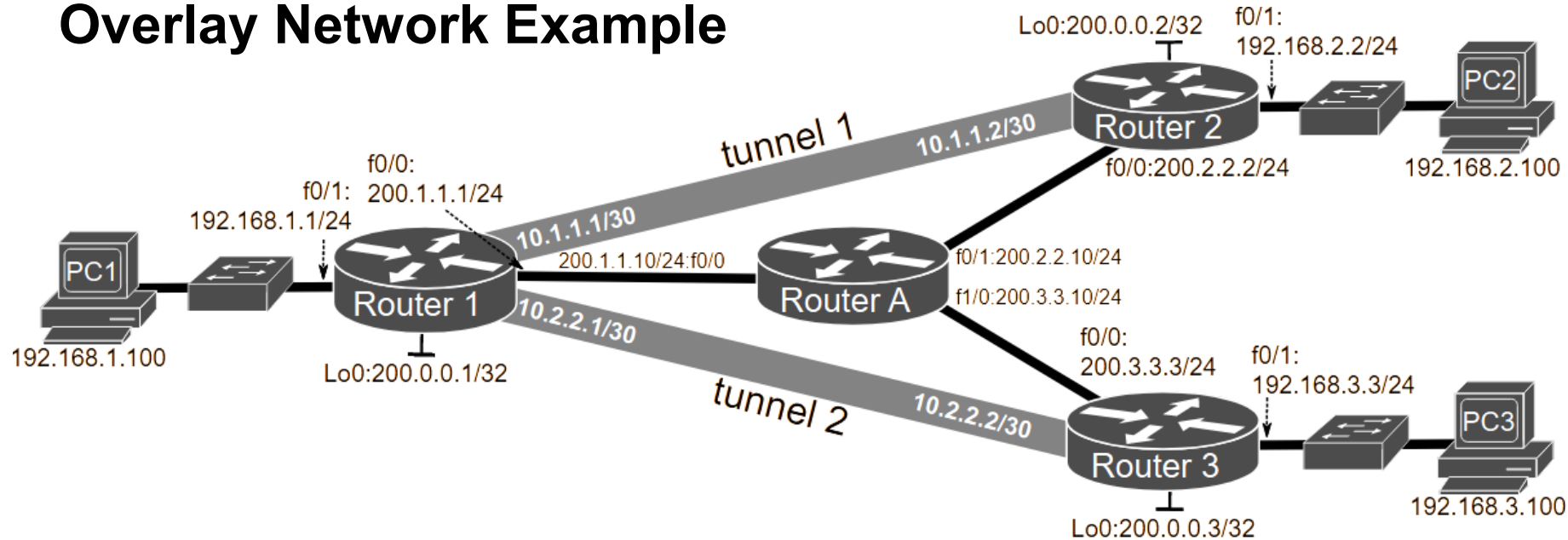
Router 1 (adds the IP networks of tunnels)

- C 10.1.1.0/30 is directly connected, Tunnel1
- C 10.2.2.0/30 is directly connected, Tunnel2
- C 192.168.1.0/24 is directly connected, F0/1
- O 200.0.0.2 [110/3] via 200.1.1.10, F0/0
- O 200.0.0.3 [110/3] via 200.1.1.10, F0/0
- C 200.1.1.0/24 is directly connected, F0/0
- O 200.2.2.0/24 [110/2] via 200.1.1.10, F0/0
- O 200.3.3.0/24 [110/2] via 200.1.1.10, F0/0

Router A (remains the same)

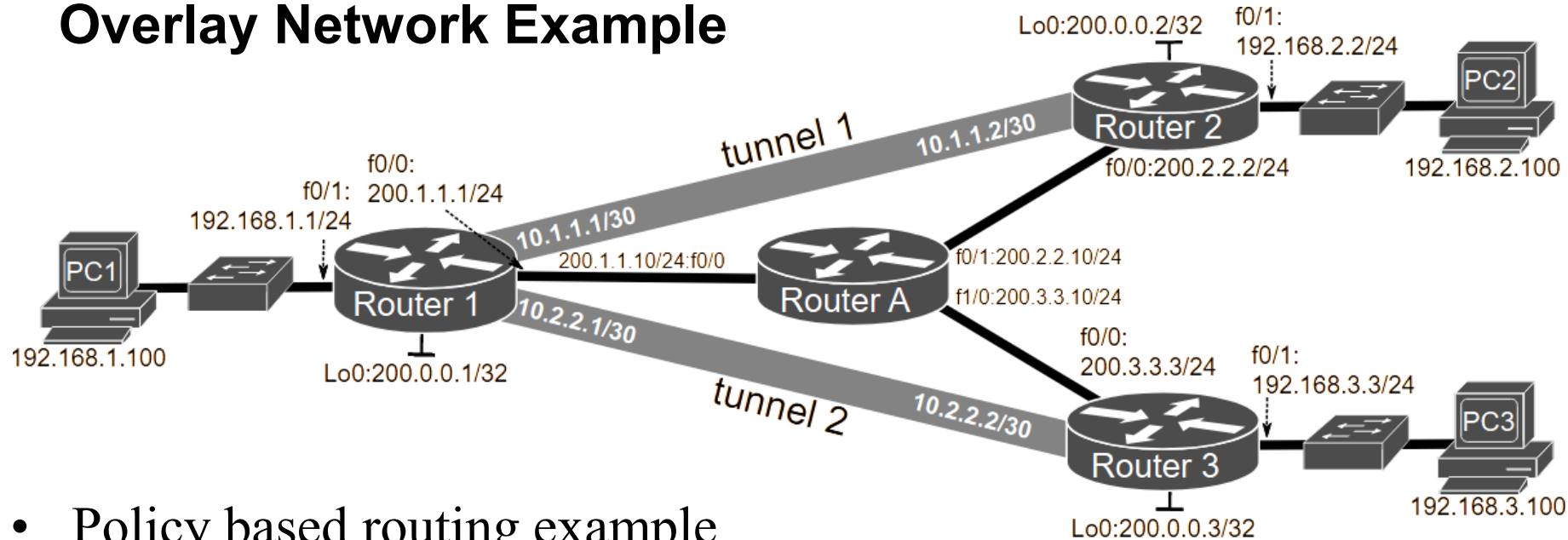
- O 200.0.0.1 [110/2] via 200.1.1.1, F0/0
- O 200.0.0.2 [110/2] via 200.2.2.2, F0/1
- O 200.0.0.3 [110/2] via 200.3.3.3, F1/0
- C 200.1.1.0/24 is directly connected, F0/0
- C 200.2.2.0/24 is directly connected, F0/1
- C 200.3.3.0/24 is directly connected, F1/0

Overlay Network Example



- As seen in the last slide, tunnels are seen as point-to-point links between end-point routers at the IP layer.
- Then, routing between the IP private networks must be established. Possible methods:
 - Policy based routing
 - Static routing
 - Dynamic routing

Overlay Network Example

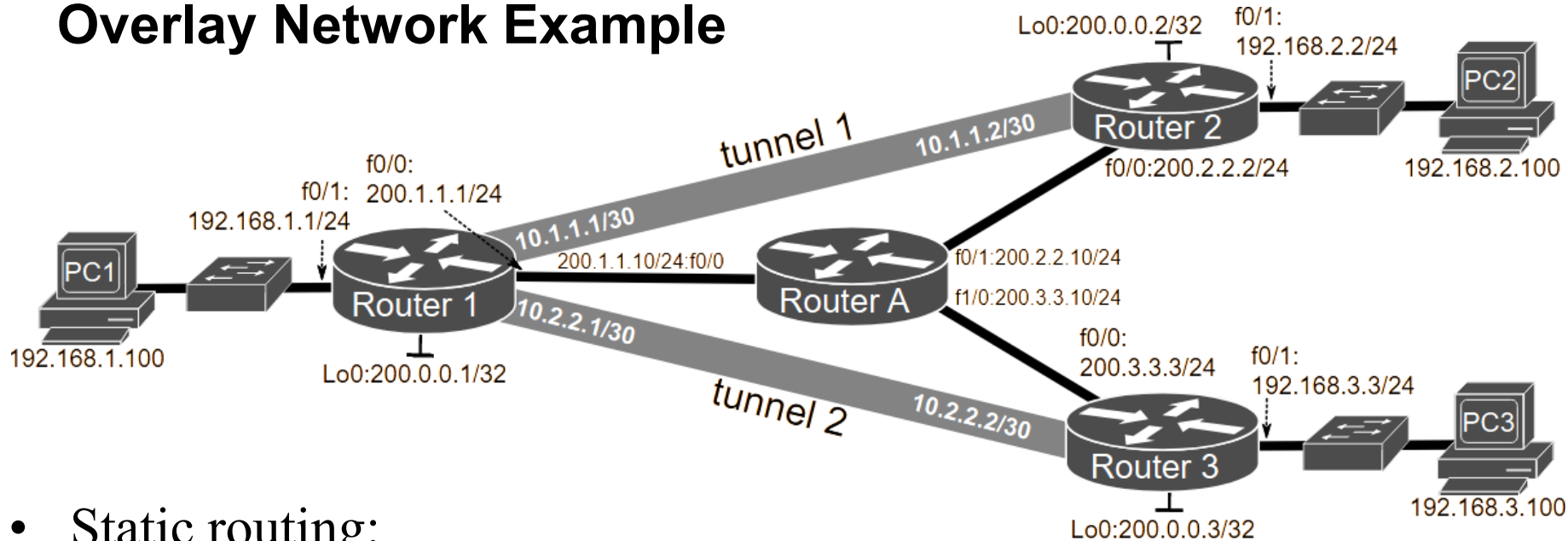


- Policy based routing example (using a route-map):

Configuration of Router 1:

```
Router1(config)# access-list 100 permit ip host 192.168.1.100 192.168.2.0 0.0.0.255
Router1(config)# route-map routeT1
Router1(config-route-map)# match ip address 100
Router1(config-route-map)# set ip next-hop 10.1.1.2
Router1(config-route-map)# interface f0/1
Router1(config-if)# ip policy route-map routeT1
```

Overlay Network Example



- Static routing:

Configuration of Router 1:

```
Router1(config)# ip route 192.168.2.0 255.255.255.0 10.1.1.2
Router1(config)# ip route 192.168.3.0 255.255.255.0 10.2.2.2
```

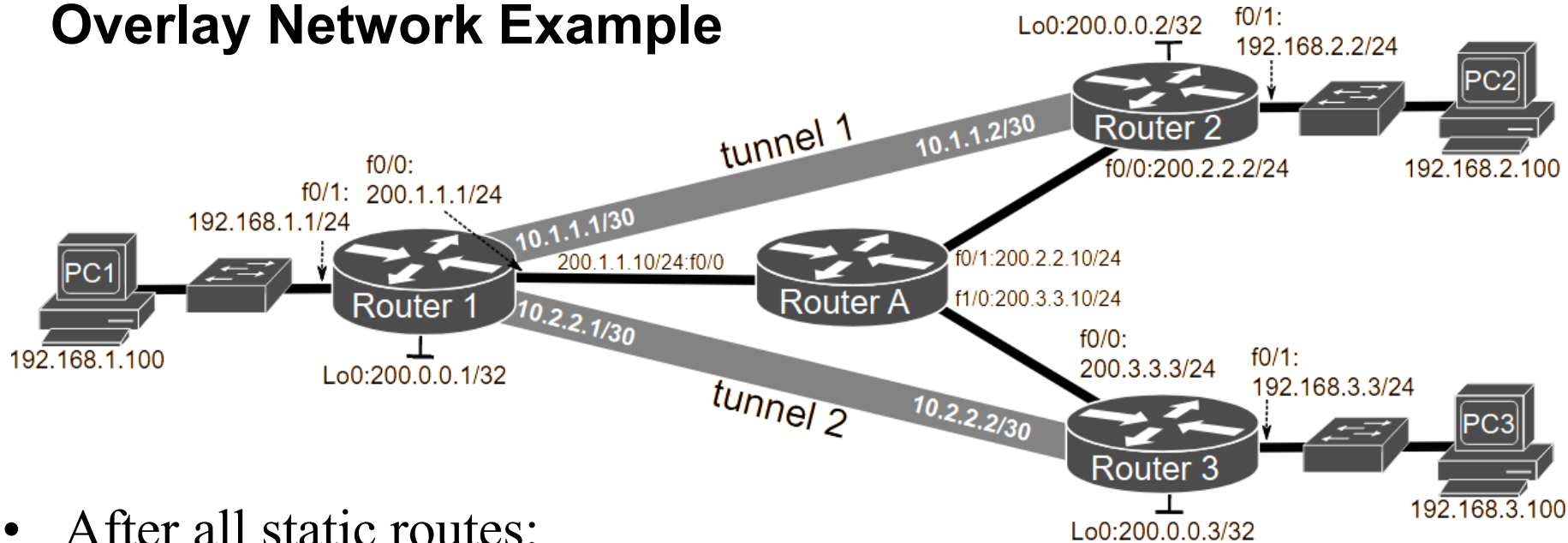
Configuration of Router 2:

```
Router2(config)# ip route 192.168.1.0 255.255.255.0 10.1.1.1
Router2(config)# ip route 192.168.3.0 255.255.255.0 10.1.1.1
```

Configuration of Router 3:

```
Router3(config)# ip route 192.168.1.0 255.255.255.0 10.2.2.1
Router3(config)# ip route 192.168.2.0 255.255.255.0 10.2.2.1
```

Overlay Network Example



- After all static routes:

Router 1 (adds the static routes)

```

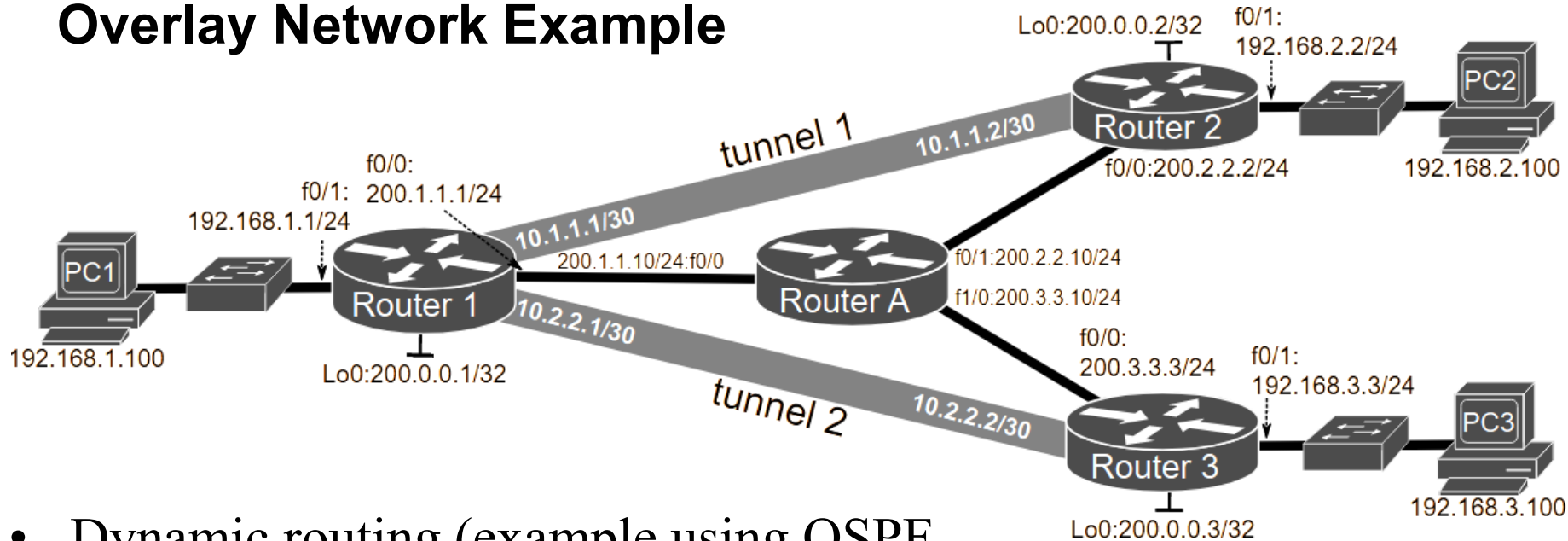
S 192.168.2.0/24 [1/0] via 10.1.1.2
S 192.168.3.0/24 [1/0] via 10.2.2.2
C 10.1.1.0/30 is directly connected, Tunnel1
C 10.2.2.0/30 is directly connected, Tunnel2
C 192.168.1.0/24 is directly connected, F0/1
O 200.0.0.2 [110/3] via 200.1.1.10, F0/0
O 200.0.0.3 [110/3] via 200.1.1.10, F0/0
C 200.1.1.0/24 is directly connected, F0/0
O 200.2.2.0/24 [110/2] via 200.1.1.10, F0/0
O 200.3.3.0/24 [110/2] via 200.1.1.10, F0/0
  
```

Router A (remains the same)

```

O 200.0.0.1 [110/2] via 200.1.1.1, F0/0
O 200.0.0.2 [110/2] via 200.2.2.2, F0/1
O 200.0.0.3 [110/2] via 200.3.3.3, F1/0
C 200.1.1.0/24 is directly connected, F0/0
C 200.2.2.0/24 is directly connected, F0/1
C 200.3.3.0/24 is directly connected, F1/0
  
```


Overlay Network Example



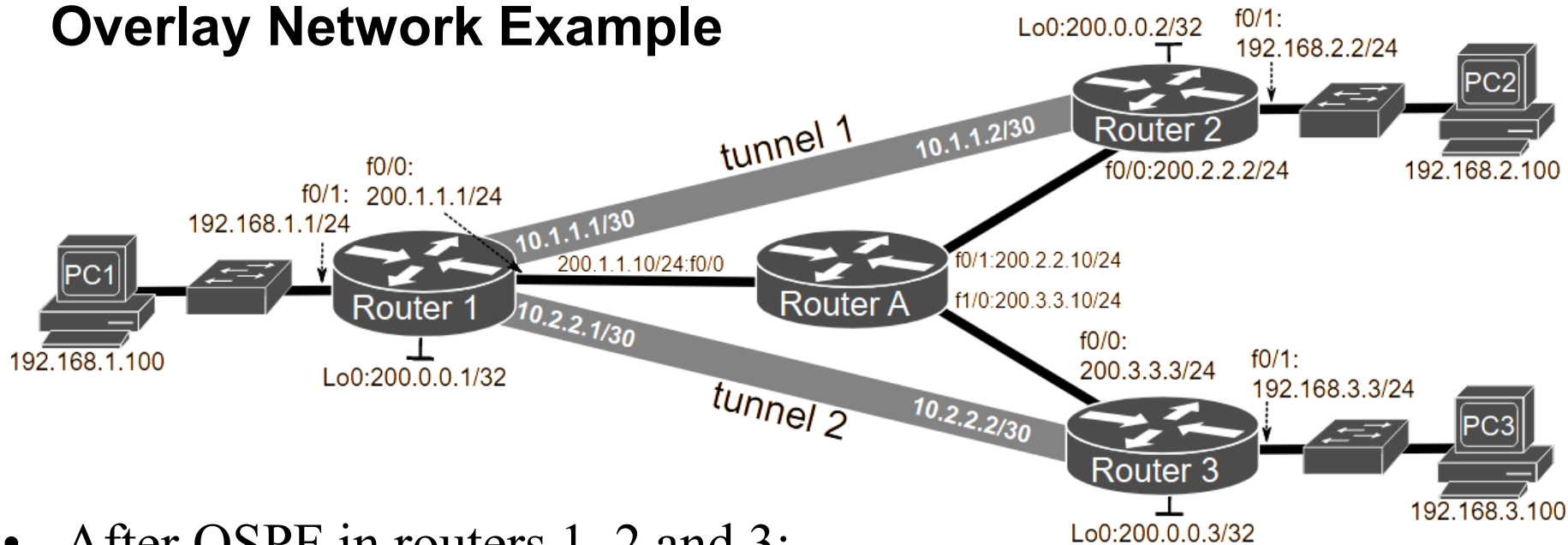
- Dynamic routing (example using OSPF with a process no. different to the one used in the public network):

Configuration of Router 1:

```
Router1(config)# interface f0/1
Router1(config-if)# ip ospf 2 area 0
Router1(config-if)# ip ospf cost 1
Router1(config-if)# interface tunnel1
Router1(config-if)# ip ospf 2 area 0
Router1(config-if)# ip ospf cost 1
Router1(config-if)# interface tunnel2
Router1(config-if)# ip ospf 2 area 0
Router1(config-if)# ip ospf cost 1
```

OSPF process no. value

Overlay Network Example



- After OSPF in routers 1, 2 and 3:

Router 1 (adds the OSPF routes)

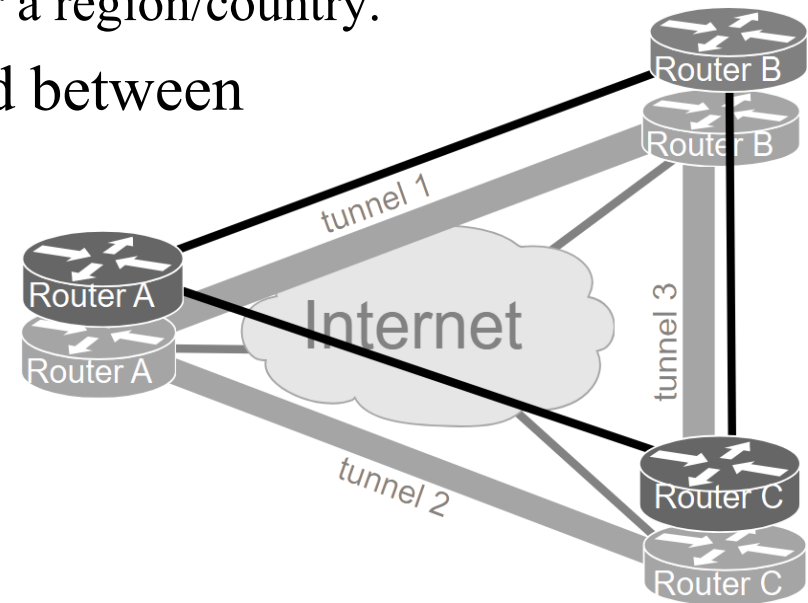
- O 192.168.2.0/24 [110/2] via 10.1.1.2, Tunnel1
- O 192.168.3.0/24 [110/2] via 10.2.2.2, Tunnel2
- C 10.1.1.0/30 is directly connected, Tunnel1
- C 10.2.2.0/30 is directly connected, Tunnel2
- C 192.168.1.0/24 is directly connected, F0/1
- O 200.0.0.2 [110/3] via 200.1.1.10, F0/0
- O 200.0.0.3 [110/3] via 200.1.1.10, F0/0
- C 200.1.1.0/24 is directly connected, F0/0
- O 200.2.2.0/24 [110/2] via 200.1.1.10, F0/0
- O 200.3.3.0/24 [110/2] via 200.1.1.10, F0/0

Router A (remains the same)

- O 200.0.0.1 [110/2] via 200.1.1.1, F0/0
- O 200.0.0.2 [110/2] via 200.2.2.2, F0/1
- O 200.0.0.3 [110/2] via 200.3.3.3, F1/0
- C 200.1.1.0/24 is directly connected, F0/0
- C 200.2.2.0/24 is directly connected, F0/1
- C 200.3.3.0/24 is directly connected, F1/0

Topology of **Overlay Networks**

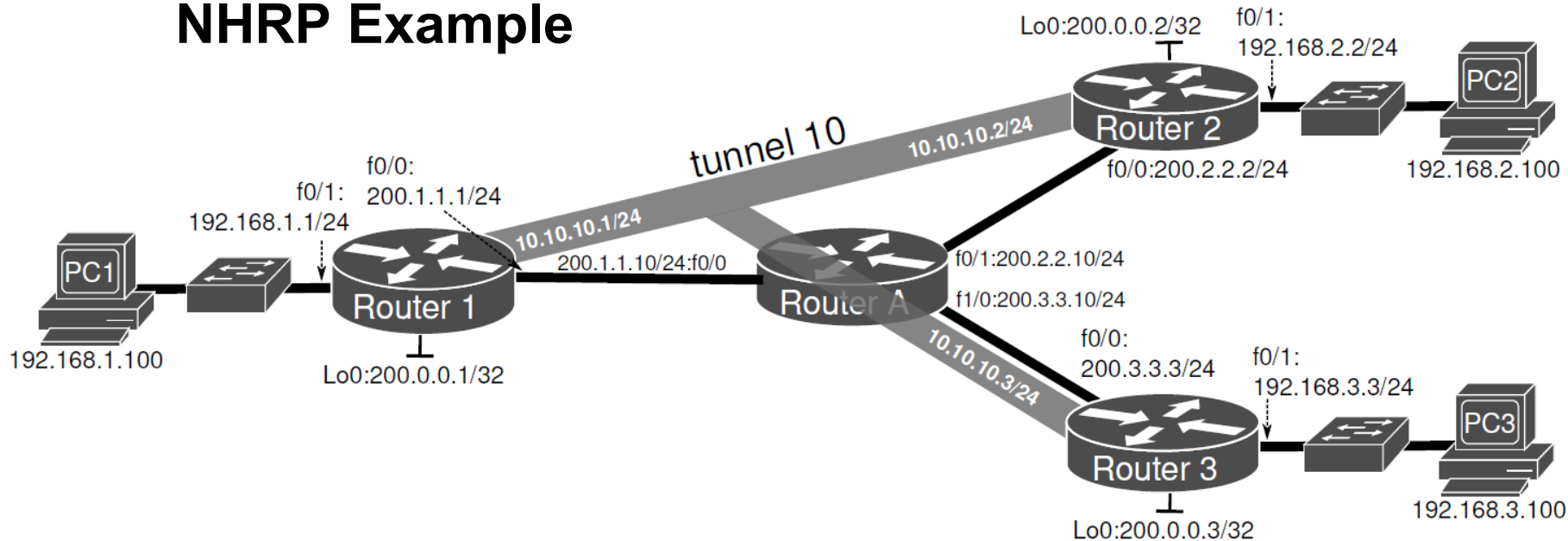
- An overlay network composed by virtual point-to-point links, each one with an assigned IP network (the case of the previous example) is a good solution when:
 - the number of involved routers is small, and
 - the IP traffic is mainly to/from a central office.
 - Example: the overlay network owner is a company with headquarters in one place and a few offices spread over a region/country.
- If the IP traffic is evenly distributed between all pairs of routers and the number of involved routers is large:
 - the topology must include a virtual point-to-point link between every pair of routers, which is not scalable.



Next Hop Resolution Protocol (NHRP)

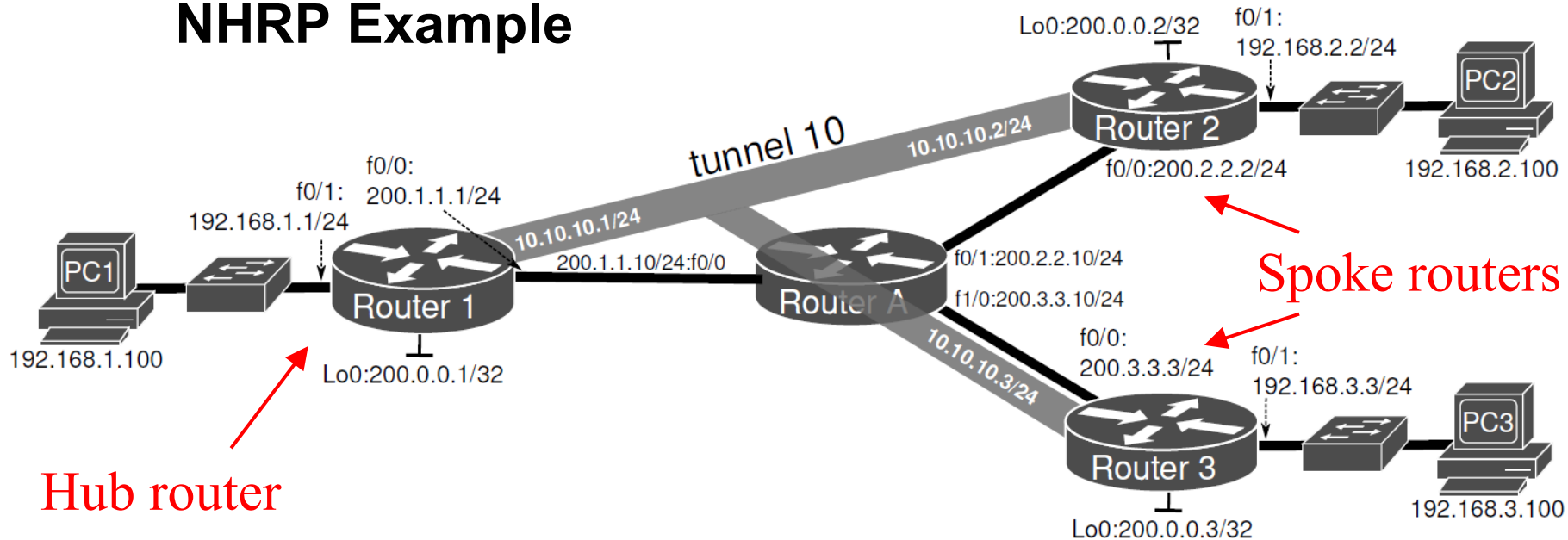
- NHRP allows the creation of a virtual network assigned with a single IP network address:
 - NHRP uses **multipoint tunnels** (instead of point-to-point tunnels)
 - In its simplest setup, one of the routers is **configured as the hub router** while all **other routers** are configured as **spoke routers**
 - The **spoke routers** register initially with the hub router (creating a hub-and-spoke network)
 - After the initial registration phase, all spoke routers know the physical/virtual IP address mapping of the hub router and the hub router knows the physical/virtual IP address mappings of all spoke routers.
 - By default, IP communications between spoke routers is conducted through the hub router.
 - If IP traffic from a sender spoke to another spoke reaches a threshold:
 - the sender spoke queries the hub to learn the physical/virtual IP address mapping of the other spoke
 - then, it sends the IP traffic directly to the other spoke.

NHRP Example



- Consider the example of the figure where:
 - The public part of the network is composed by networks 200.1.1.0/24, 200.2.2.0/24, 200.3.3.0/24 and loopback addresses (routing given by OSPF)
 - The private networks are 192.168.1.0/25, 192.168.2.0/24 and 192.168.3.0/24
 - The aim is to setup an overlay network composed by a multipoint tunnel 10 assigned with the IP network 10.10.10.0/24 to support the communications between private networks

NHRP Example



- NHRP configuration of Router 1 as Hub:

```
Router1(config)# interface Tunnel10
```

```
Router1(config-if)# ip address 10.10.10.1 255.255.255.0
```

```
Router1(config-if)# ip nhrp network-id 1
```

```
Router1(config-if)# tunnel source Loopback0
```

```
Router1(config-if)# tunnel mode gre multipoint
```

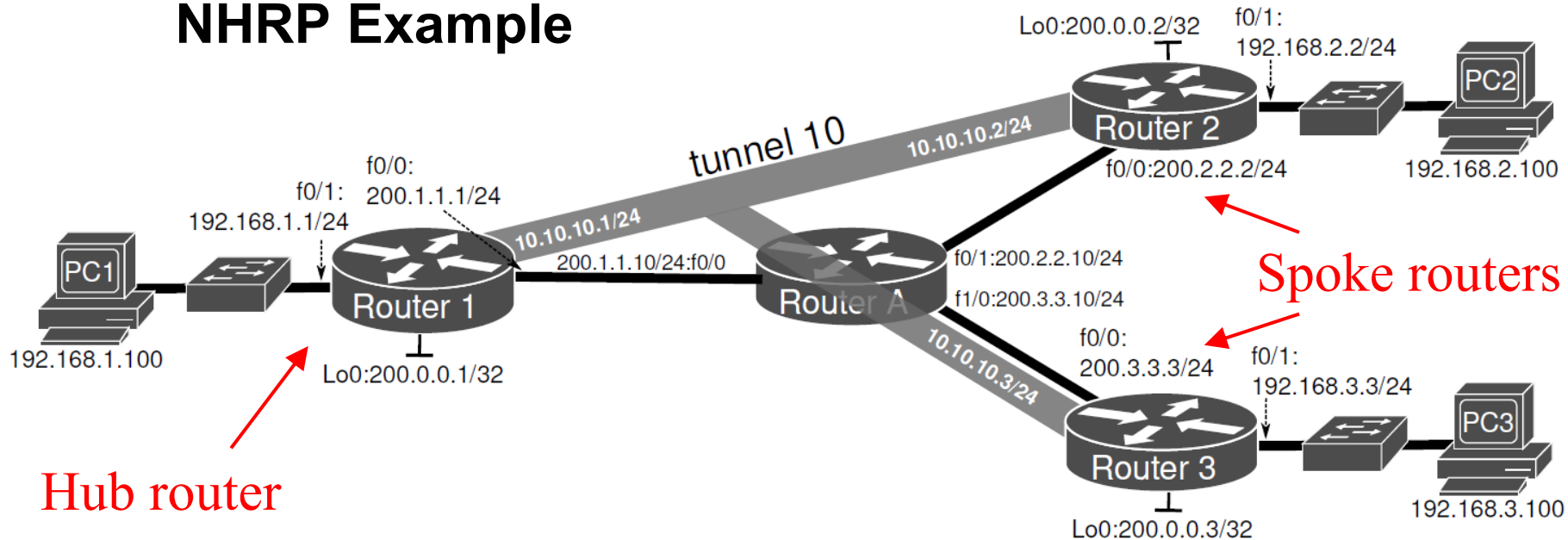
```
Router1(config-if)# tunnel key 1
```

Tunnel virtual address

Tunnel source address

Tunnel of multipoint type

NHRP Example



- NHRP configuration of Router 2 as Spoke:

```
Router2(config)# interface Tunnel10
```

```
Router2(config-if)# ip address 10.10.10.2 255.255.255.0
```

```
Router2(config-if)# ip nhrp network-id 1
```

```
Router2(config-if)# tunnel source Loopback0
```

```
Router2(config-if)# tunnel mode gre multipoint
```

```
Router2(config-if)# ip nhrp nhs 10.10.10.1
```

```
Router2(config-if)# ip nhrp map 10.10.10.1 200.0.0.1
```

```
Router2(config-if)# tunnel key 1
```

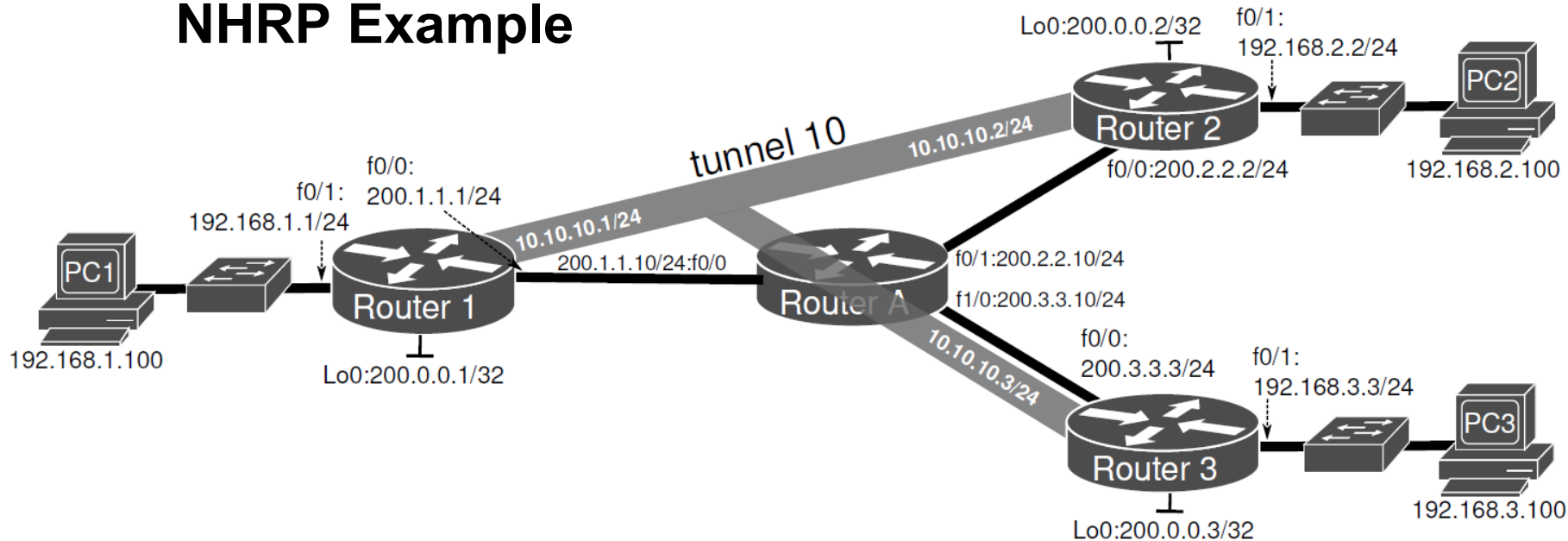
Tunnel virtual address

Tunnel source address

Tunnel of multipoint type

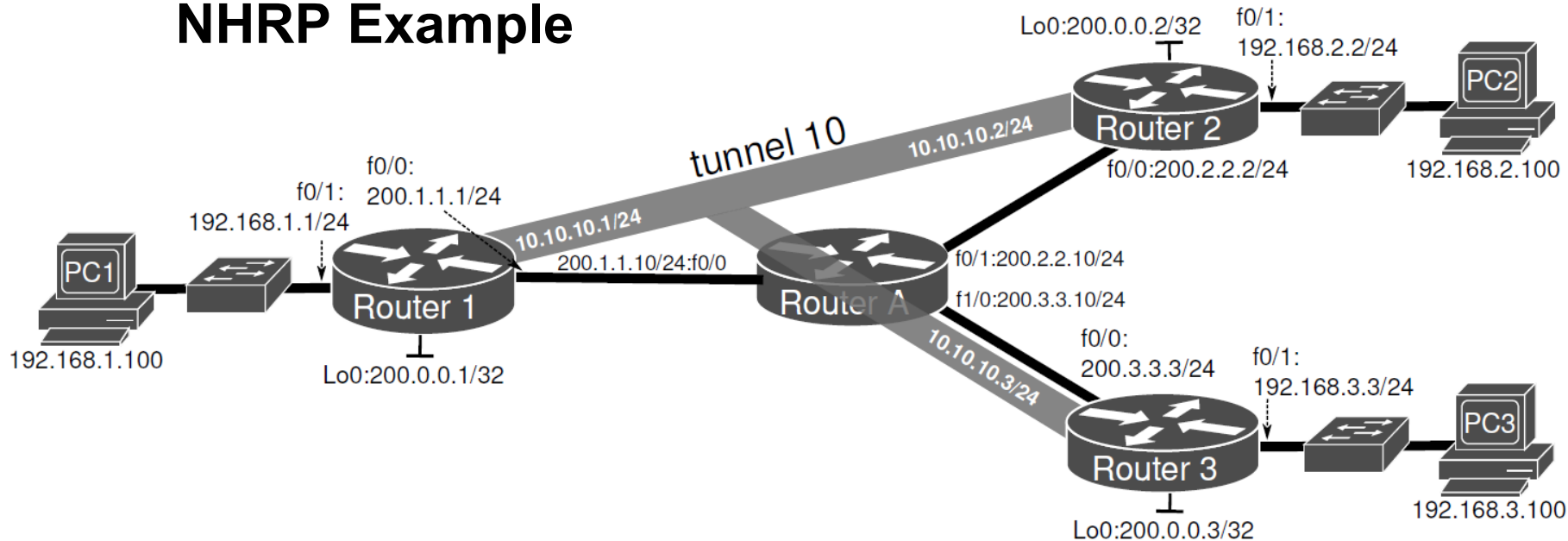
Tunnel virtual and source addresses of the Hub

NHRP Example



- After NHRP configuration and the initial registration phase:
 - Router 1 (hub router) knows that:
 - the tunnel destination address of 10.10.10.2 is 200.0.0.2
 - the tunnel destination address of 10.10.10.3 is 200.0.0.3
 - Router 2 (spoke router) knows that:
 - the tunnel destination address of 10.10.10.1 is 200.0.0.1
 - Router 3 (spoke router) knows that:
 - the tunnel destination address of 10.10.10.1 is 200.0.0.1

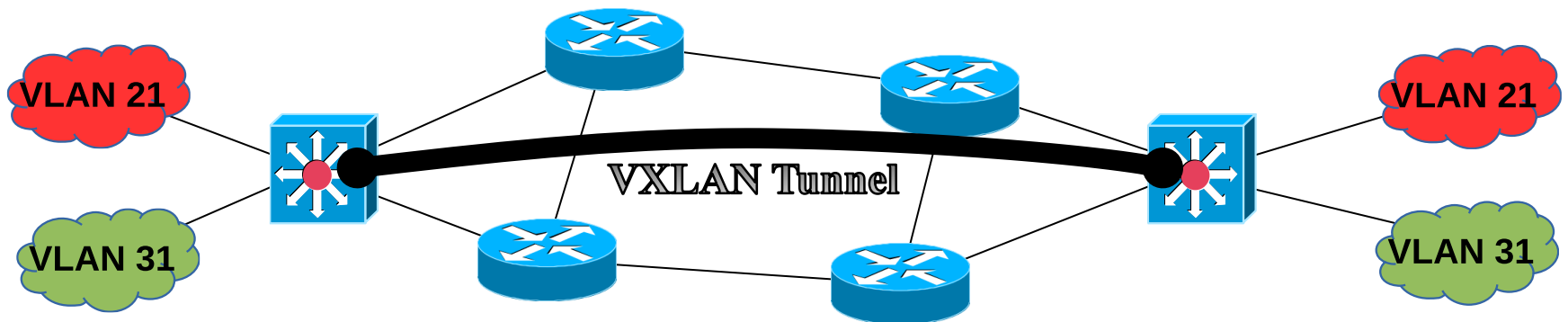
NHRP Example



- If PC2 and PC3 exchange IP packets above a given threshold:
 - Router 2 queries Router 1 (the hub) to know the tunnel destination address of the virtual address 10.10.10.3
 - Router 3 queries Router 1 (the hub) to know the tunnel destination address of the virtual address 10.10.10.2
- To minimize the number of mappings of each spoke router, the spoke-spoke mappings are dynamically:
 - setup (when there is IP traffic between them)
 - tear down (when there is no traffic between spokes)

Virtual Extensible LAN (VXLAN)

- A VLAN requires a switching domain between all hosts attached to the VLAN.
 - Recall that hosts in the same VLAN communicate at Layer 2 level.
- If VLANs are to be setup among sites connected through an IP network (Internet, for example), VXLAN provides a tunnelling solution through the IP network.

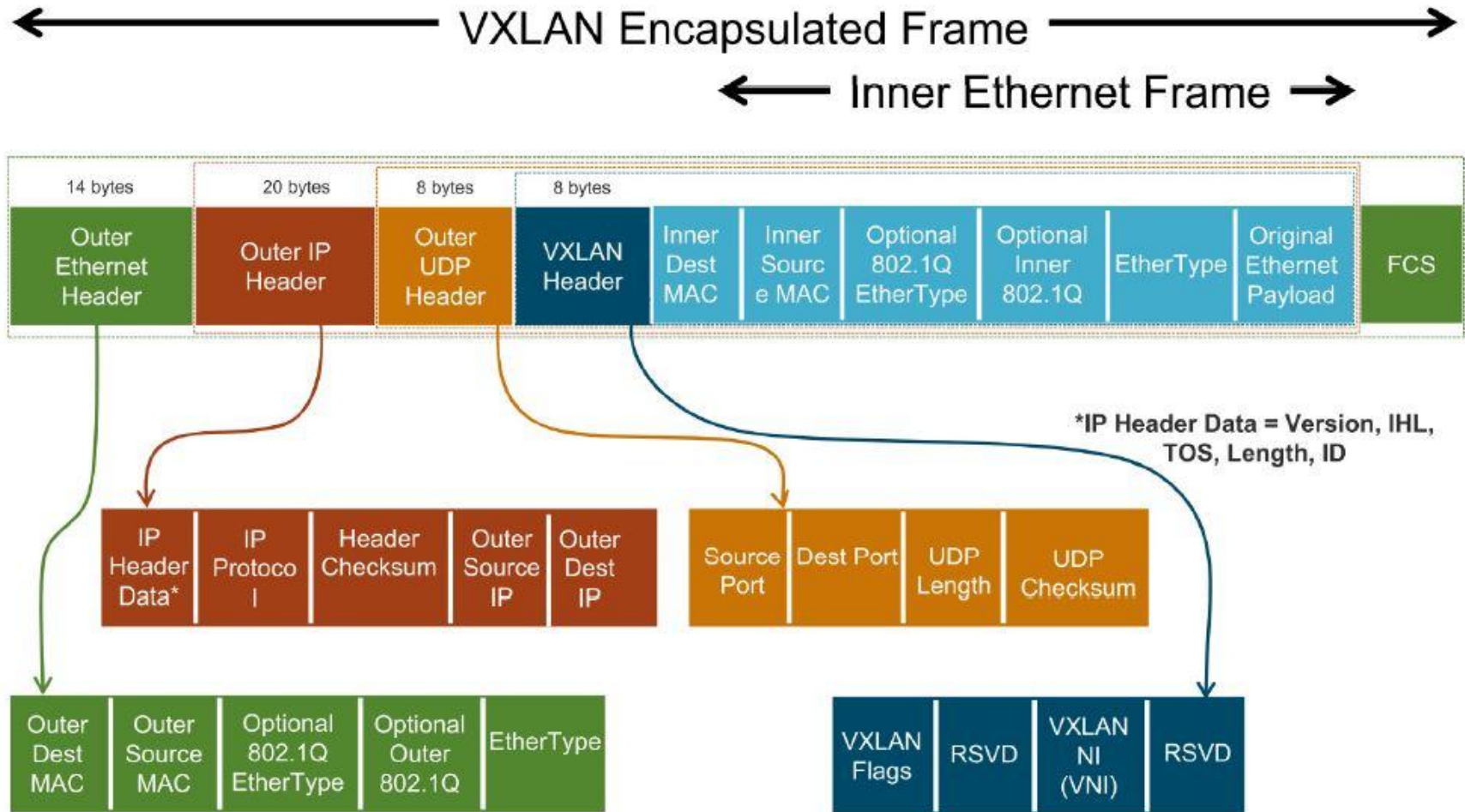


- So far, we have addressed IP (Layer 3) over IP (Layer 3) tunnelling. VXLAN provides tunnels established through IP networks and supporting Layer 2 frames

Virtual Extensible LAN (VXLAN)

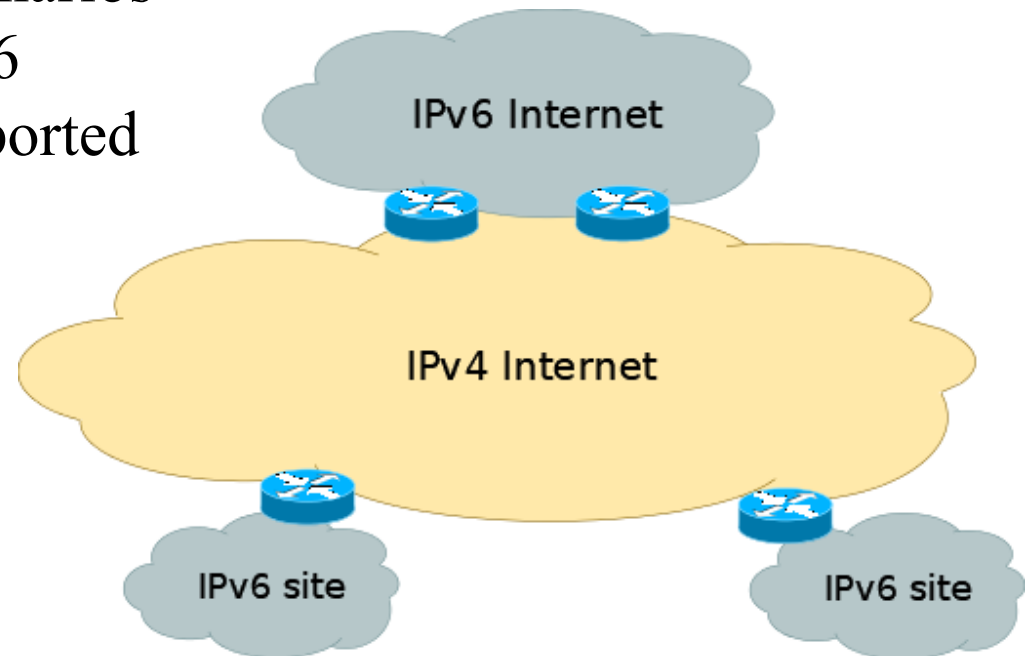
- VXLAN is a network virtualization technology designed to create virtual Layer 2 networks over existing IP (Layer 3) networks.
- It is an Alternative/Complement to 802.1Q in Layer 3 Switches.
- It encapsulates OSI Layer 2 Ethernet frames within Layer 4 UDP/IP datagrams (UDP default port 4789).
- VLAN may be additionally identified by a VNI (VXLAN Network Interface) field with 24 bits
 - 802.1Q tag only has 12 bits
 - VNIs allow for a very large number of VLANs
- VXLAN is usually used when connecting remote VLANs (connected only via IP) in Data Centres and Cloud scenarios.

Virtual Extensible LAN (VXLAN)

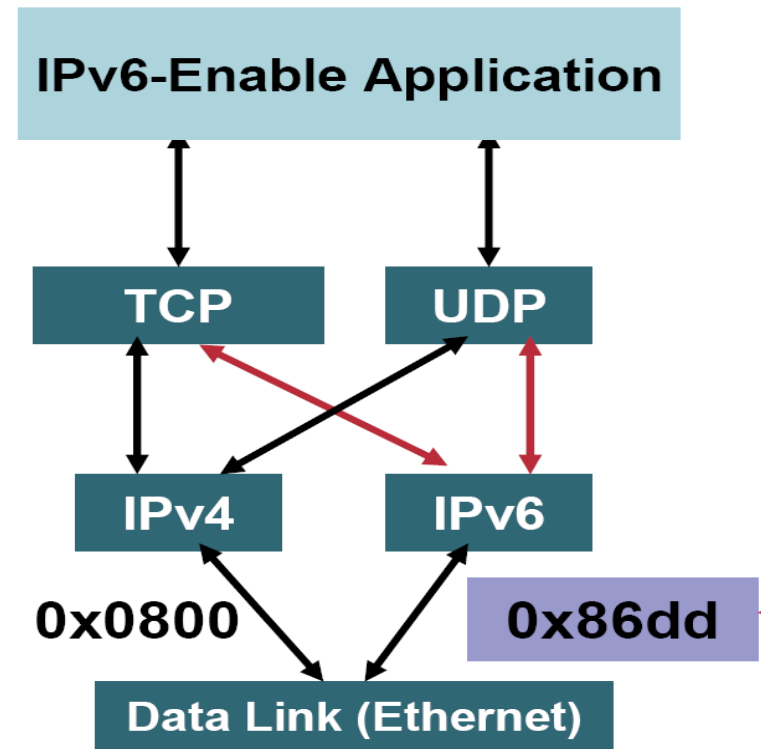
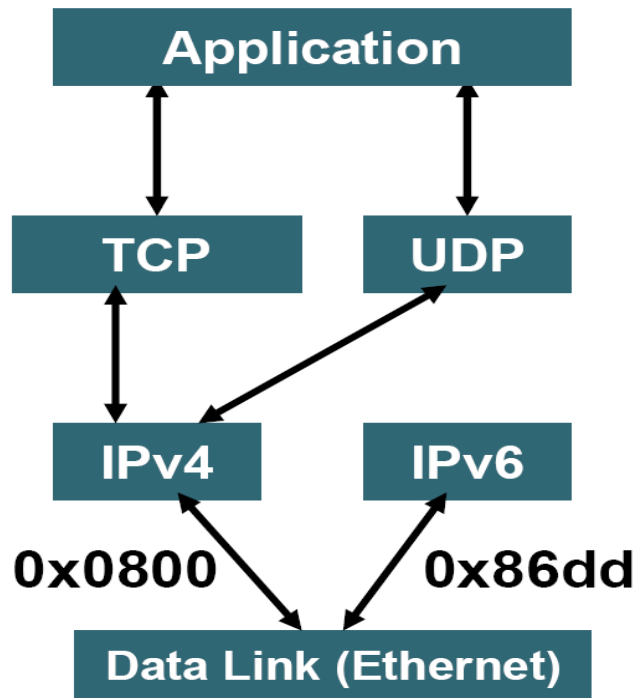


IPv6 Deployment Techniques

- The target (in the future) is to deploy IPv6 with dual-stack backbone networks
 - IPv4 and IPv6 network protocols coexist in a dual IP layer routing backbone
 - All routers on the Internet need to be upgraded to be dual-stack
- Meanwhile, transition scenarios are required in which IPv6 connectivity must be supported through IPv4 networks.



Dual Stack Hosts



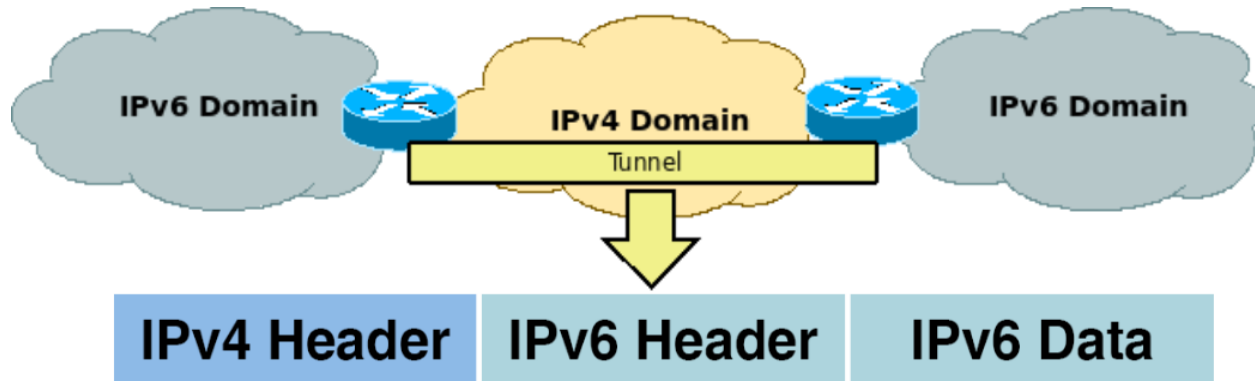
- Applications may talk through both network protocols
- Choice of the IP version is based on DNS responses and/or on application preferences

IPv6 Overlay Tunnelling in Transition Scenarios

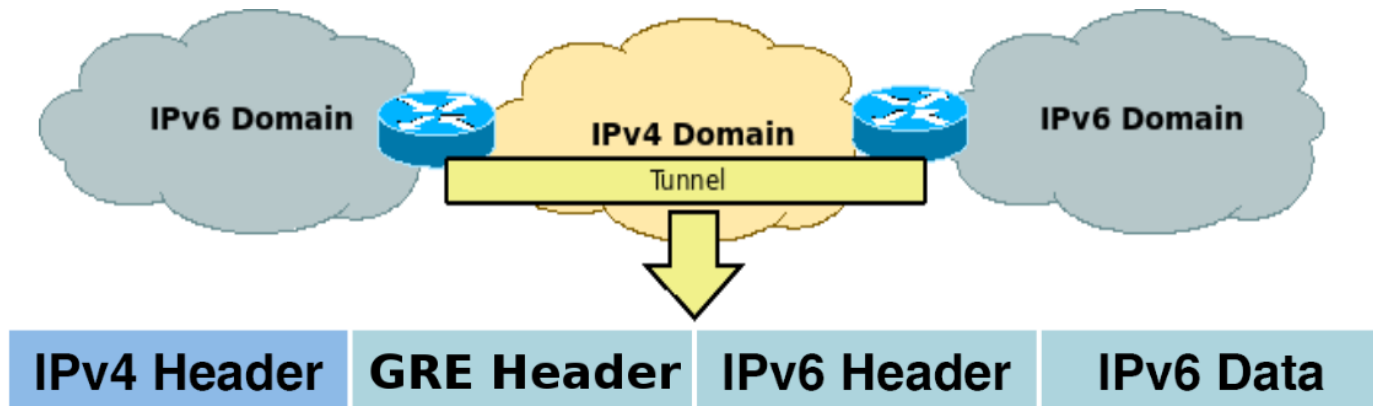
- Manual mechanisms
 - IPv6 supported through tunnel types IPv6-IPv4 or GRE IPv4
- Semi-automatic mechanisms
 - Tunnel Broker (most common implementation: Teredo)
- Automatic mechanisms
 - Common idea of automatic mechanisms:
 - the size of an IPv6 address (16 bytes) is much larger than the size of an IPv4 address (4 bytes)
 - embedding IPv4 addresses into the IPv6 addresses enables the automatic determination of the tunnel exit end-point IPv4 addresses
 - Examples:
 - 6to4 Tunnels
 - ISATAP Tunnels

IPv6 over Manually Configured Tunnels

- IPv6-IPv4 tunnel type:

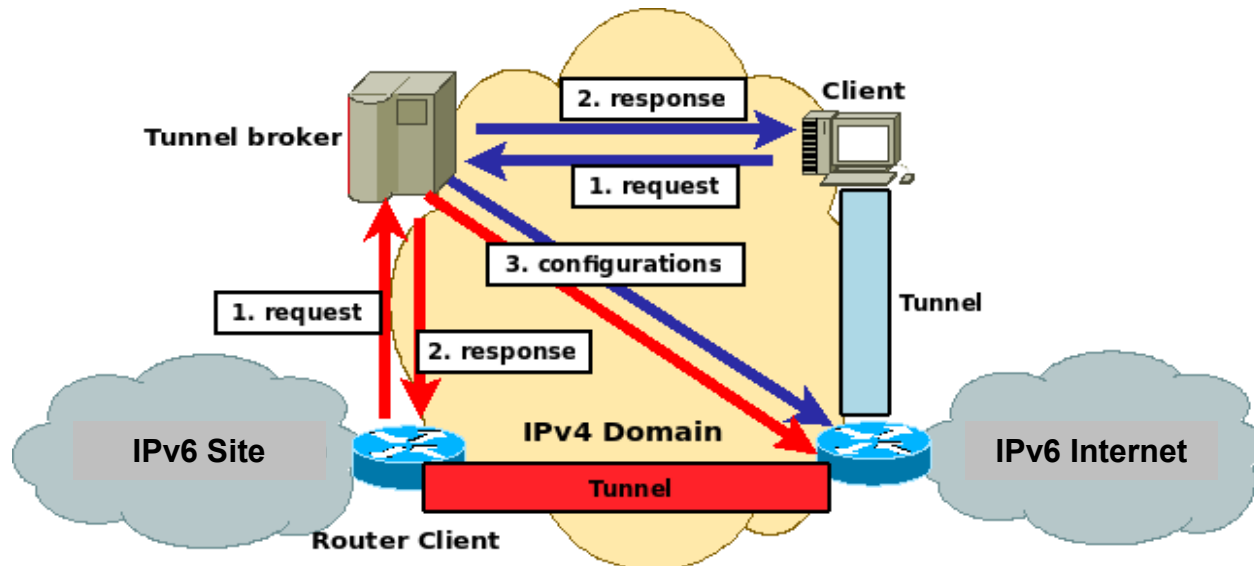


- GRE IPv4 tunnel type:



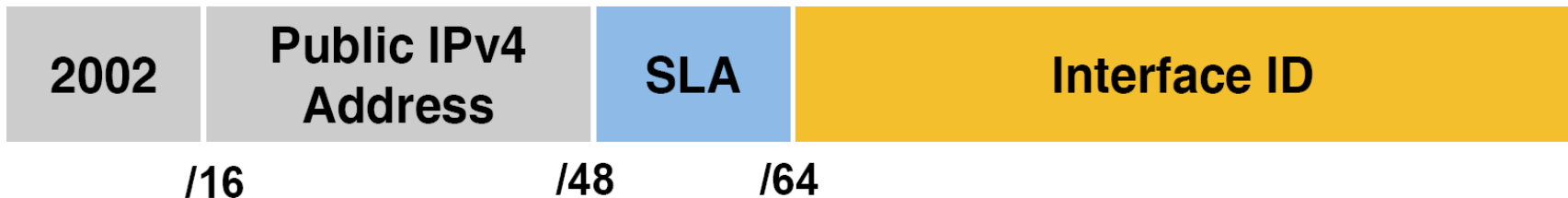
Tunnel Broker

- A tunnel broker service allows IPv6 applications on dual-stack systems access to an IPv6 backbone through a IPv4 network
- Clients can be behind NAT boxes
- The Broker manages tunnel requests and configuration
- Potential security issue as the broker is a single point of failure
- Most common implementation: Teredo.
 - The Teredo IPv6 address block is 2001:0:XXXX:XXXX::/64 where XXXX:XXXX is the public IPv4 address of the broker



6to4 Tunnels

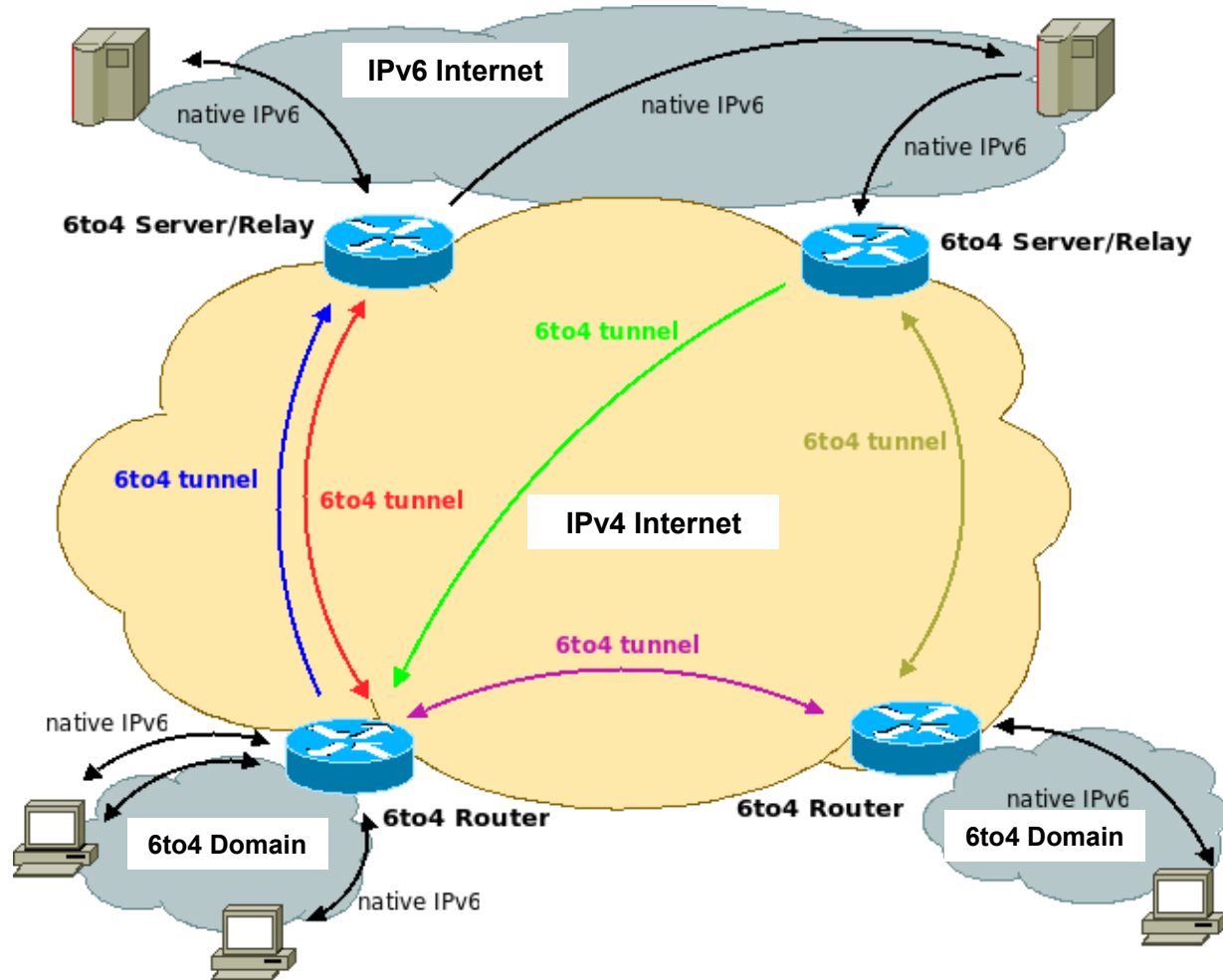
- Automatic 6to4 tunnels allow isolated IPv6 domains to connect (between them or with the IPv6 Internet) over an IPv4 network
- IPv4 tunnel end-point address is embedded within the 6to4 IPv6 address:



- 6to4 hosts/routers need to have a globally addressable IPv4 address:
 - The IPv4 address cannot be a NAT address (i.e., the host/router cannot be located behind a NAT box) unless NAT supports protocol 41 packets forwarding
 - The value 41 in the protocol field of an IPv4 header identifies that payload of the IPv4 packet is an IPv6 packet

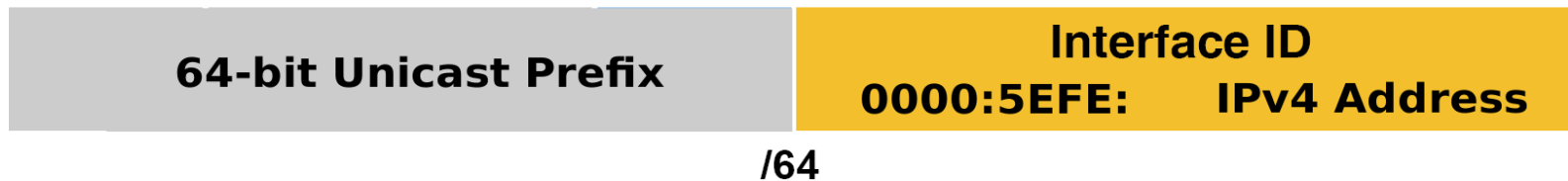
6to4 Relay Routers

- 6to4 router connects hosts of its 6to4 domain with:
 - hosts of other 6to4 domains through their 6to4 routers
 - hosts in the IPv6 Internet through a 6to4 relay router
- 6to4 relay router connects hosts on the IPv6 Internet with:
 - hosts of 6to4 domains through their 6to4 routers



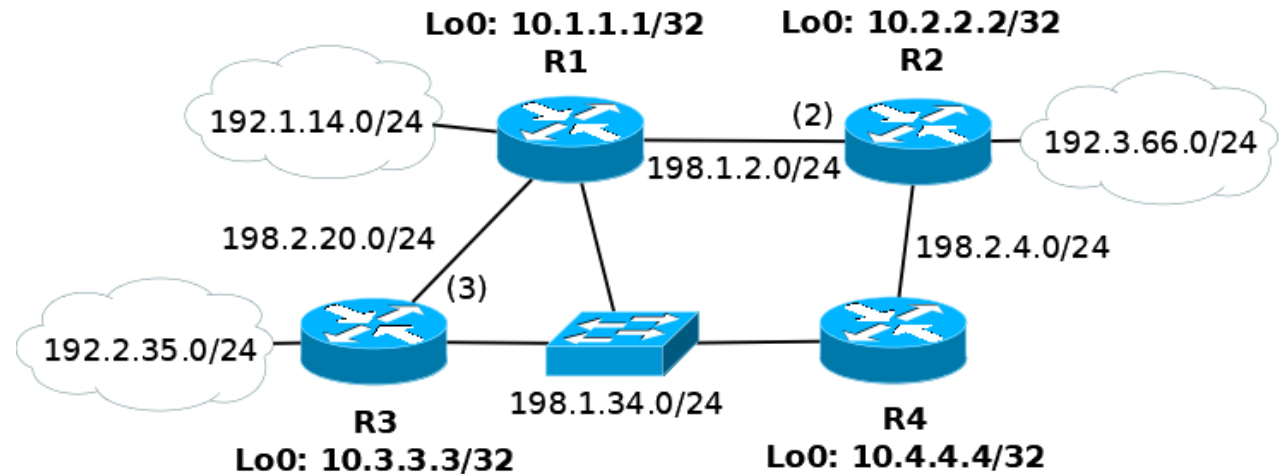
ISATAP Tunnels

- ISATAP (Intra-site Automatic Tunnel Address Protocol)
 - is an automatic overlay tunnelling mechanism that uses the underlying IPv4 network as a non-broadcast multiple access link layer for IPv6.
 - is designed for transporting IPv6 packets within a site where a native IPv6 infrastructure is not yet available
- It encodes IPv4 Address in IPv6 Address within the interface ID:



- ISATAP is designed to support IPv6 communications within a IPv4 site, and not between multiple sites.
- The unicast IPv6 prefix (/64) is announced by the ISATAP router. It can be link local, site local or global (including 6to4 prefixes), enabling IPv6 routing locally or towards the IPv6 Internet.

Example



Consider the IP network of the figure where each router has an IP address assigned to its Loopback0 interface. In the physical interfaces of routers, the host part of each IP address is the number of the router's name. All routers are configured with OSPF in all IP networks (the OSPF costs that are not equal to 1 are defined in the figure between brackets.) There is one tunnel configured between the loopback interfaces of R2 and R3. A static route is also configured in R3 to reach 192.1.14.0/24 through the tunnel.

- What are the OSPF entries of the IP routing table of R3?
- What is the sequence of routers crossed by the ICMP Echo Requests and ICMP Echo Replies in a ping from a host in 192.1.14.0/24 to a host in 192.2.35.0/24?
- Indicate (and justify) if the IP communications between network 192.2.35.0/24 and network 192.3.66.0/24 are routed through the configured tunnel.